

# Variáveis Compostas Heterogêneas

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

# Introdução

- Até agora, variáveis armazenam apenas **um único tipo** de dado
  - Definido na declaração
- Variáveis servem, também, para representar entidades identificadas no problema real que será resolvido computacionalmente
- É bastante comum a necessidade de armazenar, dentro de uma mesma variável, **diferentes tipos** de dados
- É aí que entram em cena os **registros**

# Registros

- **Registros** conseguem agregar vários dados acerca de uma mesma entidade
- Programadores podem gerar **novos tipos** de dados, não se limitando à utilização dos tipos de dados primitivos
- Cada dado contido em um registro é chamado **campo**
  - Os **campos** podem ser de diferentes tipos primitivos ou, ainda, podem representar outros **registros**
- É por essa razão que os registros são conhecidos, também, como **variáveis compostas heterogêneas**

# Definição de Estrutura

- Os registros em C/C++, chamados de **estruturas**, são definidos por meio da utilização da palavra reservada **struct**
- A partir da estrutura definida, o programa poderá considerar que existe um **novο tipo de dado**

```
struct nome_da_estrutura {  
    <tipo> <campo1>;  
    <tipo> <campo2>;  
    ...  
    <tipo> <campoN>;  
};
```

# Exemplo 1: Defina Struct Banco

```
struct banco {  
    int num;  
    char titular[35];  
    float saldo;  
};
```

- Uma estrutura chamada **banco** foi definida

# Declaração de Estrutura

- A declaração de variáveis desse tipo, é feita da seguinte forma:

```
struct nome_da_estrutura nome_da_variável;
```

- Estruturas representam **novos tipos** de dados
  - Todas as operações e declarações realizadas com os tipos predefinidos da linguagem também poderão ser realizadas com as estruturas

# Exemplo 2: Declare Struct Banco

```
#include <stdio.h>
...
struct banco {
    int num;
    char titular[35];
    float saldo;
};
...
int main() {
    ...
    struct banco conta;
    ...
}
```



# Representação Variável Conta

Variável conta

|         |  |
|---------|--|
| num     |  |
| titular |  |
| saldo   |  |

- Assim, conta tem três campos: **num**, **titular** e **saldo**



# Exemplo 3: Matriz de Contas

```
#include <stdio.h>
...
struct banco {
    int num;
    char titular[35];
    float saldo;
};
...
int main() {
    ...
    struct banco contas[4][3];
    ...
}
```

- Observe que **contas** é uma matriz com 4 linhas e 3 colunas. Cada posição dessa matriz terá espaço para armazenar três valores: **num**, **titular** e **saldo**

# Representação Variável Contas

|   | 0       |  | 1       |  | 2       |  |
|---|---------|--|---------|--|---------|--|
| 0 | num     |  | num     |  | num     |  |
|   | titular |  | titular |  | titular |  |
|   | saldo   |  | saldo   |  | saldo   |  |
| 1 | num     |  | num     |  | num     |  |
|   | titular |  | titular |  | titular |  |
|   | saldo   |  | saldo   |  | saldo   |  |
| 2 | num     |  | num     |  | num     |  |
|   | titular |  | titular |  | titular |  |
|   | saldo   |  | saldo   |  | saldo   |  |
| 3 | num     |  | num     |  | num     |  |
|   | titular |  | titular |  | titular |  |
|   | saldo   |  | saldo   |  | saldo   |  |

# Acesso aos Campos da Estrutura

- O uso de uma **struct** é possível por meio do acesso individual a seus **campos**, quer seja para gravar quer seja para recuperar um dado
- O acesso a determinado campo da estrutura é feito informando-se o nome da variável, seguido por um ponto e pelo nome do campo desejado:

`nome_da_variável_do_tipo_estrutura.nome_do_campo`

# Exemplo 4: Acesso aos Campos

```
#include <stdio.h>

struct banco {
    int num;
    char titular[35];
    float saldo;
};

int main() {
    struct banco conta;

    printf("Digite o numero da conta: ");
    scanf("%d%c", &conta.num);
    printf("Digite o nome do titular da conta: ");
    scanf("%34s%c", conta.titular);
    printf("Digite o saldo da conta: ");
    scanf("%f%c", &conta.saldo);

    return 0;
}
```

# Exemplo 5: Vetor de Struct

```
#include <stdio.h>
...
struct empresa {
    char nome[50];
    float salario;
};
...
int main() {
    ...
    struct empresa funcionarios[4];
    ...
    for(i = 0; i < 4; i++) {
        printf("Digite o nome do funcionário %d : ", i);
        scanf("%49s%c", funcionarios[i].nome);
        printf("Digite o salário do funcionário %d : ", i);
        scanf("%f%c", &funcionarios[i].salario);
    }
    ...
}
```

# Exemplo 5: Vetor de Struct (Cont.)

- Cada posição desse vetor tem capacidade para armazenar os campos **nome** e **salário**, definidos na estrutura **empresa**
- Assim, como acontece com qualquer vetor, as posições devem ser acessadas com o uso de um índice

# Representação Vetor Funcionários

|   |         |         |
|---|---------|---------|
| 0 | nome    | João    |
|   | salario | 1000,00 |
| 1 | nome    | Maria   |
|   | salario | 5000,00 |
| 2 | nome    | Pedro   |
|   | salario | 1800,00 |
| 3 | nome    | Lúcia   |
|   | salario | 2700,00 |



# Exemplo 6: Acesso i-ésimo Struct

```
...  
for(i = 0; i < 4; i++) {  
    printf("\nFuncionario que ocupa a posicao %d no vetor:", i);  
    printf("\nNome: %s", funcionarios[i].nome);  
    printf("\nSalario: %6.2f", funcionarios[i].salario);  
    printf("\n");  
}  
...
```

- A estrutura de repetição **for**, permite que as quatro posições do vetor sejam percorridas e os dados encontrados sejam mostrados



## Exemplo 6: Acesso i-ésimo Struct (Cont.)

Funcionario que ocupa a posicao 0 no vetor:

Nome: Joao

Salario: 1000.00

Funcionario que ocupa a posicao 1 no vetor:

Nome: Maria

Salario: 5000.00

Funcionario que ocupa a posicao 2 no vetor:

Nome: Pedro

Salario: 1800.00

Funcionario que ocupa a posicao 3 no vetor:

Nome: Lucia

Salario: 2700.00

# Exemplo 7: Typedef

```
#include <stdio.h>

typedef struct {
    char nome[20];
    char sobrenome[20];
    int idade;
    char sexo;
    double salario;
} funcionario;

int main() {
    funcionario f = {"Marie", "Silva", 30};
    printf("%s %s (%d anos)\n", f.nome, f.sobrenome, f.idade);
    return 0;
}
```

Palavra reservada

O nome deve vir para o final

Não é mais necessário colocar struct antes do nome

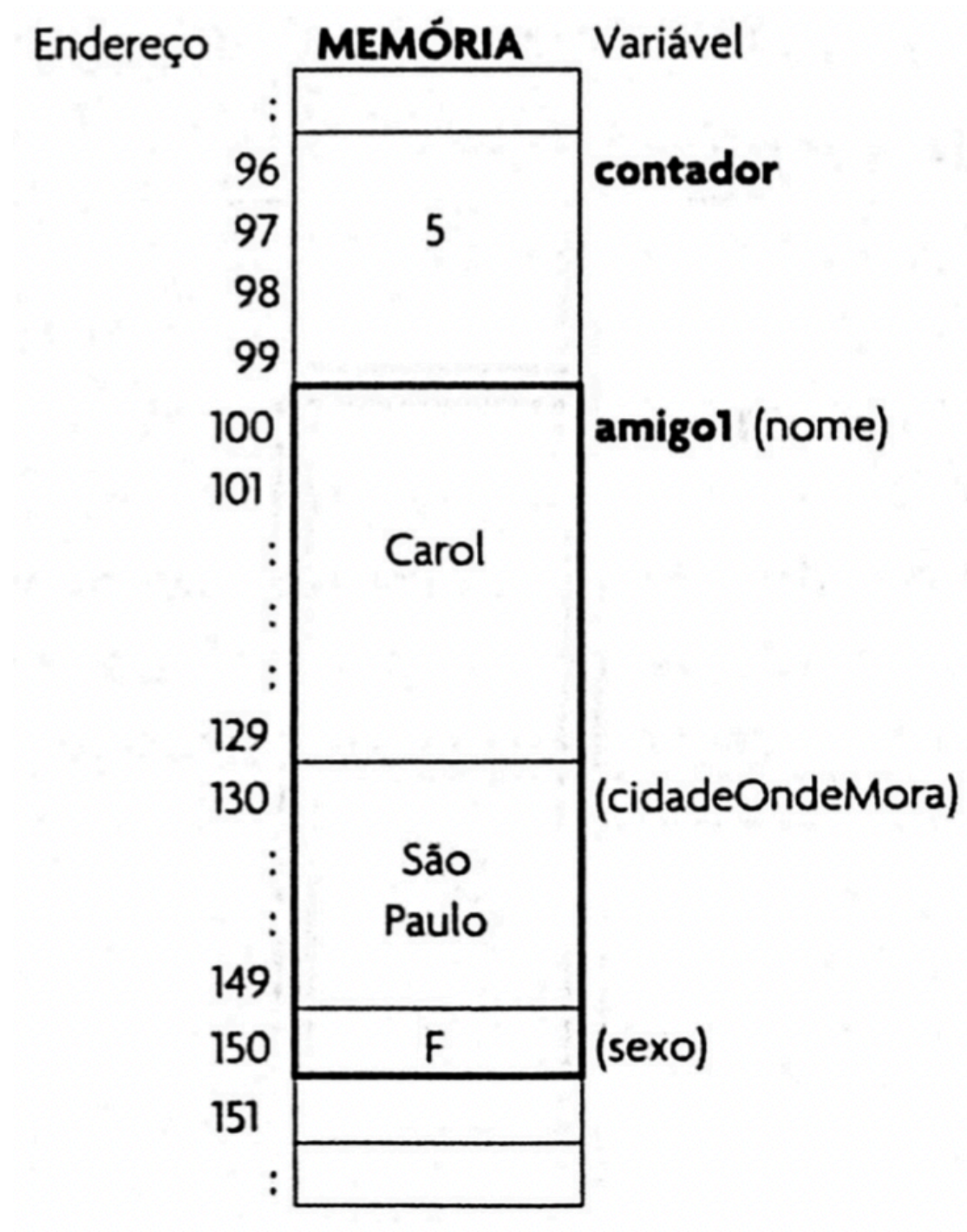
# Palavra Reservada Typedef

- Declarações com **typedef** não produzem novos tipos de dados. Criam apenas novos nomes (sinônimos) para os tipos existentes

# Representação na Memória

- Vamos supor a declaração de duas variáveis:
- Variável `int contador`, ocupa quatro bytes. Foi atribuído o valor 5.
- Variável `struct amigo1` com três atributos:
  - `char nome[30]`, ocupa 30 bytes e possui valor “Carol”
  - `char cidadeOndeMora[20]`, ocupa 20 bytes e possui valor “São Paulo”
  - `char sexo`, ocupa 1 byte e possui o valor ‘F’ (feminino)

# Representação na Memória (2)



# Representação na Memória (3)

- Registro **amigo1** ocupa 51 bytes (30+20+1)
  - Endereços de memória de 100 a 150
- Quando **contador** for acessada (operação de leitura ou escrita), serão acessados quatro bytes a partir do primeiro byte da variável **contador**, isto é, a partir do endereço de memória 96
- Quando **amigo1** for acessada, serão acessados 51 bytes (tamanho do registro) a partir do primeiro byte dela, isto é, a partir do endereço 100

# Constantes de Enumeração



# Constantes

- É tudo que é fixo, imutável ...
- Pode ser um número, um caractere ou ou uma sequência qualquer de caracteres (string)
- Exemplos:
  - Numérica: 15, -15, 0.342, -2726
  - Caractere: 'a', 'b', 'c'
  - Literal: "José da Silva", "123456", "\*A!B?-“, "16/08/2010"



# Constantes: #define

```
#include <stdio.h>
```

```
#define ANO 2025
```

```
#define UNIVERSIDADE "Universidade Federal de Mato Grosso do Sul"
```

```
int main () {  
    printf("Ano: %d\n", ANO);  
    printf("Curso: %s\n", UNIVERSIDADE);  
  
    return 0;  
}
```

# Constantes: const

```
/* Exemplo uso de const para declarar constantes */
#include <stdio.h>

int main () {
    const char BIP = '\a'; // declaracao de constante
    const double PI = 3.14159265358; // declaracao de constante
    double raio, area;

    printf("Digite o raio da esfera: ");
    scanf("%lf", &raio);
    area = PI * raio * raio;

    printf("%c%c", BIP, BIP); // BIP BIP
    printf("\nArea da esfera = %lf", area);

    return 0;
}
```

# Constantes: enum

- **enum** são conhecidos como tipos enumerados
- Consistem num **conjunto de constantes inteiras**, em que cada uma delas é representada por um nome
- A variável desse tipo é sempre **int** e, para cada um dos valores do conjunto, atribuímos um nome significativo

```
enum cor {  
    VERMELHO,  
    VERDE,  
    AZUL,  
    AMARELO,  
};
```

# Tipo Enumerados: enum

- A vantagem é que utilizamos esses nomes no lugar de números, o que torna o programa mais claro
- A palavra **enum** enumera a lista de nomes automaticamente, dando-lhes números em sequência (0, 1, 2, etc.)
- Caso queira, é possível atribuir valores explícitos:

```
enum cor {  
    VERMELHO = 1,  
    VERDE = 2,  
    AZUL = 3,  
    AMARELO = 4,  
};
```

```
enum cor {  
    VERMELHO = 1,  
    VERDE,  
    AZUL,  
    AMARELO,  
};
```

# Exemplo 8: enum mes

```
...
enum mes {JAN=1, FEV, MAR, ABR, MAI, JUN, JUL, AGO, SET, OUT, NOV, DEZ};

int main() {
    // Cria duas variáveis do tipo enum mês
    enum mes m1, m2;

    // Atribui valores
    m1 = ABR;
    m2 = JUN;

    // Operações aritméticas permitidas
    m3 = m2 - m1;

    // Comparações permitidas
    if (m1 < m2)
        ...
}
```

# Problema: Obter Situação Aluno

- Considere a seguinte estrutura aluno

```
struct aluno {  
    float prova_1, prova_2, prova_substitutiva, exame;  
    int faltas;  
};
```

# Exemplo 9: Sem enum

```
int obter_situacao_aluno (struct aluno a) {
    int flag;
    float media_aproveitamento, p1 = a.prova_1, p2 = a.prova_2;

    if(a.faltas > 17)
        flag = 2; // reprovado por falta
    else {
        if(a.prova_1 < a.prova_2 && a.prova_1 < a.prova_optativa)
            media_aproveitamento = (a.prova_optativa + a.prova_2) / 2;
        else if (a.prova_2 < a.prova_1 && a.prova_2 < a.prova_optativa)
            media_aproveitamento = (a.prova_1 + a.prova_optativa) / 2;
        else
            media_aproveitamento = (a.prova_1 + a.prova_2) / 2;

        if(media_aproveitamento < 6.0)
            flag = 1; // reprovado
    }
}
```



```

int obter_situacao_aluno (struct aluno a) {
    int flag;
    float media_aproveitamento, p1 = a.prova_1, p2 = a.prova_2;

    if(a.faltas > 17)
        flag = 2; // reprovado por falta
    else {
        if(a.prova_1 < a.prova_2 && a.prova_1 < a.prova_optativa)
            media_aproveitamento = (a.prova_optativa + a.prova_2) / 2;
        else if (a.prova_2 < a.prova_1 && a.prova_2 < a.prova_optativa)
            media_aproveitamento = (a.prova_1 + a.prova_optativa) / 2;
        else
            media_aproveitamento = (a.prova_1 + a.prova_2) / 2;

        if(media_aproveitamento < 6.0)
            flag = 1; // reprovado
        else
            flag = 0; // aprovado
    }
    return flag;
}

```



# Exemplo 10: Com enum

```
enum situacao_aluno {
    APROVADO, REPROVADO, REPROVADO_POR_FALTA};
...
enum situacao_aluno obter_situacao_aluno (struct aluno a) {
    enum situacao_aluno situacao;
    float media_aproveitamento;

    if(a.n_faltas > 17)
        situacao = REPROVADO_POR_FALTA;
    else {
        if(a.prova_1 < a.prova_2 && a.prova_1 < a.prova_optativa)
            media_aproveitamento = (a.prova_optativa + a.prova_2) / 2;
        else if (a.prova_2 < a.prova_1 && a.prova_2 < a.prova_optativa)
            media_aproveitamento = (a.prova_1 + a.prova_optativa) / 2;
        else
            media_aproveitamento = (a.prova_1 + a.prova_2) / 2;

        if(media_aproveitamento < 6.0)
            situacao = REPROVADO;;
    }
}
```

```

enum situacao_aluno {
    APROVADO, REPROVADO, REPROVADO_POR_FALTA};
...
enum situacao_aluno obter_situacao_aluno (struct aluno a) {
    enum situacao_aluno situacao;
    float media_aproveitamento;

    if(a.n_faltas > 17)
        situacao = REPROVADO_POR_FALTA;
    else {
        if(a.prova_1 < a.prova_2 && a.prova_1 < a.prova_optativa)
            media_aproveitamento = (a.prova_optativa + a.prova_2) / 2;
        else if (a.prova_2 < a.prova_1 && a.prova_2 < a.prova_optativa)
            media_aproveitamento = (a.prova_1 + a.prova_optativa) / 2;
        else
            media_aproveitamento = (a.prova_1 + a.prova_2) / 2;

        if(media_aproveitamento < 6.0)
            situacao = REPROVADO;;
        else
            situacao = APROVADO;
    }
    return situacao;
}

```

# Exercício em Sala

Crie um programa que defina um novo tipo estrutura chamado *ponto2d*. Essa estrutura deve ser capaz de armazenar os valores das coordenadas cartesianas  $x$  e  $y$ , representando um ponto no espaço bidimensional. Escreva uma função *dist\_euclidiana* que possua dois parâmetros do tipo *ponto2d*, e calcula e retorna a distância dos pontos. A fórmula da distância euclidiana é dada abaixo:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Obs.: Você precisará adaptar essa fórmula para trabalhar com as duas estruturas *ponto2d* recebidas pela função. Obs. 2.: A raiz quadrada pode ser calculada usando a função *sqrt*, presente na biblioteca *math*.

# Referências

- ASCENCIO, A. F. G.; CAMPOS, E. A. V. Fundamentos da programação de computadores: algoritmos, Pascal, C/C++ e Java. 3. ed. São Paulo: Pearson, 2012.
- DEITEL, P.; DEITEL, H. C: Como Programar. 6a ed. São Paulo: Pearson, 2011.
- PIVA, D.J. et al. Algoritmos e programação de computadores. Rio de Janeiro: Elsevier, 2012.